



Fanout в RabbitMQ

1. Введение

Мы уже построили REST-сервис и научили его отправлять асинхронные уведомления через RabbitMQ. Также мы отдельно изучили gRPC. Теперь пришло время объединить эти технологии в единую экосистему, как это делается в реальных архитектурах крупных компаний.

В этом занятии мы реализуем **гибридное взаимодействие**:

Синхронное (gRPC)

Наш основной сервис demo-rest будет обращаться к новому сервису аналитики для быстрого расчета данных. Здесь важна скорость и строгая типизация, поэтому используем gRPC.

Асинхронное (RabbitMQ Fanout)

После получения расчета мы опубликуем событие, которое должно быть доставлено **всем** заинтересованным сервисам. Для этого мы сменим тип обменника на Fanout.

В сложных системах (Uber, Netflix) один сервис редко делает всё сам.

Пользователь запрашивает отчет -> Основной сервис идет в Analytics Service по **gRPC** (потому что это быстро и внутри кластера), получает цифры -> Основной сервис публикует событие ReportGenerated. **Подписчики:**

- **Notification Service** отправляет Push-уведомление пользователю.
- **Billing Service** списывает деньги за отчет.
- **Audit Service** (или сам основной сервис) пишет лог в историю.

Чтобы все три сервиса получили одно и то же сообщение, нам нужен **Fanout**. Если бы мы использовали одну очередь на всех, сообщение досталось бы только одному счастливчику.

Необходимое ПО:

1. Проекты demo-rest, audit-service.
2. Нарботки по gRPC из предыдущей работы (мы превратим их в analytics-service).
3. Docker с RabbitMQ.



2. Теория

gRPC в микросервисной архитектуре

gRPC не заменяет REST/GraphQL, он их дополняет.

- **REST/GraphQL** - внешний интерфейс для фронтенда/мобильных клиентов (публичный контракт).
- **gRPC** – общение «сервер-сервер» внутри закрытого контура (Private Network). Низкая задержка, бинарный протокол, строгие контракты.

Fanout Exchange (вещание)

В прошлый раз мы использовали Topic Exchange. Он фильтрует сообщения по ключу.

Fanout Exchange — «громкоговоритель». Он игнорирует Routing Key.

Логика его работы такая: «Я получил сообщение. Я отправлю его копию во **ВСЕ** очереди, которые ко мне привязаны».

Очереди подписчиков (Consumer Queues)

Это критически важный момент.

Конкурирующие потребители (Worker Pattern)

Если NotificationService запущен в 5 экземплярах и все слушают **одну** очередь - сообщение обработает только **один** экземпляр. Это балансировка нагрузки.

Pub/Sub (Broadcasting)

Если мы хотим, чтобы событие обработал и NotificationService, и AuditService - у каждого сервиса должна быть **СВОЯ** очередь, привязанная к обменнику.

3. Практический разбор

1. **Создаем analytics-service (gRPC сервер), который считает «рейтинг пользователя»**
2. **В demo-rest (gRPC клиент) добавляем эндпоинт, который вызывает аналитику**
3. **Получив рейтинг, demo-rest отправляет событие UserRatedEvent в Fanout обменник**
4. **audit-service слушает свою очередь (шлет уведомление)**
5. **demo-rest тоже слушает это событие через отдельную очередь (для логирования/кеширования), демонстрируя принцип Fanout**



Подготовка analytics-service (gRPC Сервер)

Возьмите проект grpc-server из предыдущей работы за основу и создайте analytics-service.

1. Контракт (analytics.proto)

2. В папке src/main/proto создайте файл analytics.proto.

```
syntax = "proto3";
```

```
option java_multiple_files = true;
```

```
option java_package = "grpc.demo";
```

```
service AnalyticsService {  
    rpc      CalculateUserRating      (UserRatingRequest)      returns  
(UserRatingResponse) {}  
}
```

```
message UserRatingRequest {  
    int64 user_id = 1;  
    string category = 2;  
}
```

```
message UserRatingResponse {  
    int64 user_id = 1;  
    int32 rating_score = 2;  
    string verdict = 3;  
}
```

Выполните сборку, чтобы сгенерировать Java-классы.

3. Реализация сервиса (AnalyticsServiceImpl):

```
@GrpcService  
public      class      AnalyticsServiceImpl      extends  
AnalyticsServiceGrpc.AnalyticsServiceImplBase {
```



```
@Override
public void calculateUserRating(UserRatingRequest request,
StreamObserver<UserRatingResponse> responseObserver) {
    // Имитация бурной деятельности и сложных расчетов
    int score = (int) (Math.random() * 100);
    String verdict = score > 50 ? "GOOD" : "BAD";

    UserRatingResponse response = UserRatingResponse.newBuilder()
        .setUserId(request.getUserId())
        .setRatingScore(score)
        .setVerdict(verdict)
        .build();

    responseObserver.onNext(response);
    responseObserver.onCompleted();
}
}
```

Запустите analytics-service (по умолчанию порт 9090).

[Интеграция клиента в demo-rest](#)

Нам нужно научить наш REST-сервис ходить в gRPC.

Зависимости

Добавьте в pom.xml проекта demo-rest те же зависимости для gRPC клиента и плагин protobuf, что были в прошлой работе.

Скопируйте файл analytics.proto в src/main/proto внутри demo-rest. В реальной жизни это было бы отдельной библиотекой, но пока копируем. *Не забудьте пересобрать для demo-rest!*

1. В application.properties (demo-rest):

```
grpc.client.analytics-service.address=static://localhost:9090
grpc.client.analytics-service.negotiation-type=plaintext
```

2. Добавьте в общий модуль events-contract новое событие.

```
// events/UserRatedEvent.java
```



```
public record UserRatedEvent(Long userId, Integer score, String verdict)
implements Serializable {}
```

Пересоберите events-contract, потом обновите эту сборку во всех проектах.

3. Конфигурация RabbitMQ (Fanout). Обновите RabbitMQConfig.java в demo-rest.

@Configuration

```
public class RabbitMQConfig {
```

```
    public static final String EXCHANGE_NAME = "books-exchange";
```

```
    public static final String ROUTING_KEY_BOOK_CREATED =
```

```
"book.created";
```

```
    public static final String FANOUT_EXCHANGE = "analytics-fanout";
```

@Bean

```
    public FanoutExchange analyticsExchange() {
```

```
        return new FanoutExchange(FANOUT_EXCHANGE, true, false);
```

```
    }
```

@Bean

```
    public TopicExchange booksExchange() {
```

```
        return new TopicExchange(EXCHANGE_NAME);
```

```
    }
```

@Bean

```
    public MessageConverter jsonMessageConverter() {
```

```
        return new Jackson2JsonMessageConverter();
```

```
    }
```

@Bean

```
    public RabbitTemplate rabbitTemplate(ConnectionFactory
connectionFactory) {
```

```
        RabbitTemplate template = new
```

```
RabbitTemplate(connectionFactory);
```



```
        template.setMessageConverter(jsonMessageConverter());  
        return template;  
    }  
}
```

```
@Bean  
public SimpleRabbitListenerContainerFactory  
rabbitListenerContainerFactory(  
    ConnectionFactory connectionFactory) {  
    SimpleRabbitListenerContainerFactory factory = new  
SimpleRabbitListenerContainerFactory();  
    factory.setConnectionFactory(connectionFactory);  
    factory.setMessageConverter(jsonMessageConverter());  
    return factory;  
}  
}
```

4. Добавьте метод в любой сервис или контроллер demo-rest.

```
@RestController  
public class RatingController {  
  
    @GrpcClient("analytics-service")  
    private AnalyticsServiceGrpc.AnalyticsServiceBlockingStub  
analyticsStub;  
  
    private final RabbitTemplate rabbitTemplate;  
  
    public RatingController(RabbitTemplate rabbitTemplate) {  
        this.rabbitTemplate = rabbitTemplate;  
    }  
  
    @PostMapping("/api/users/{id}/rate")  
    public String rateUser(@PathVariable Long id) {  
        // Вызов gRPC  
    }  
}
```



| Fanout

```
var request =
UserRatingRequest.newBuilder().setUserId(id).setCategory("General").build();

var gRpcResponse = analyticsStub.calculateUserRating(request);

// Отправка события в Fanout
var event = new UserRatedEvent(gRpcResponse.getUserId(),
gRpcResponse.getRatingScore(), gRpcResponse.getVerdict());

// Для Fanout routingKey не важен, оставляем пустым ""
rabbitTemplate.convertAndSend(RabbitMQConfig.FANOUT_EXCHANGE,
"", event);

return "Rating calculated: " + gRpcResponse.getRatingScore();
}
}
```

Настройка подписчиков (fanout)

Главная магия Fanout - каждому подписчику нужна **своя уникальная очередь**.

1. Подписчик 1: audit-service

В BookEventListener.java (или создайте новый Listener) добавьте подписку на Fanout.

```
@RabbitListener(
    bindings = @QueueBinding(
        // Уникальное имя очереди для уведомлений!
        value = @Queue(name = "q.audit.analytics", durable = "true"),
        exchange = @Exchange(name = "analytics-fanout", type = "fanout")
    )
)
public void handleRating(UserRatedEvent event) {
    log.info("NOTIFY: Sending email. User {} has new rating: {}",
event.userId(), event.score());
}
```

2. demo-rest (сам себе слушатель)



Давайте добавим слушателя прямо в demo-rest, чтобы доказать, что сообщение дублируется.

```
// listeners/InternalAnalyticsListener
```

```
@Component
```

```
public class InternalAnalyticsListener {
```

```
    @RabbitListener(
```

```
        bindings = @QueueBinding(
```

```
            // Нужно другое имя очереди!
```

```
            value = @Queue(name = "q.demorest.analytics.log", durable =  
"true"),
```

```
            exchange = @Exchange(name = "analytics-fanout", type =  
"fanout")
```

```
        )
```

```
    )
```

```
    public void logRating(UserRatedEvent event) {
```

```
        System.out.println("We just rated user " + event.userId());
```

```
    }
```

```
}
```

Разрешаем и пакет "edu.rutmiit.demo.events" в Main классе DemoRestApplication!

```
@SpringBootApplication(
```

```
    scanBasePackages = {"edu.rutmiit.demo.demorest",
```

```
"edu.rutmiit.demo.booksapicontract", "edu.rutmiit.demo.events"},
```

```
    exclude = {DataSourceAutoConfiguration.class}
```

```
)
```

```
@EnableHypermediaSupport(type =
```

```
EnableHypermediaSupport.HypermediaType.HAL)
```

```
public class DemoRestApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(DemoRestApplication.class, args);
```



```
}
```

```
}
```

3. audit-service (обновим конфиг)

```
@Configuration
public class RabbitMQConfig {

    @Bean
    public MessageConverter messageConverter() {
        return new Jackson2JsonMessageConverter();
    }

    @Bean
    public RabbitTemplate rabbitTemplate(ConnectionFactory
connectionFactory) {
        RabbitTemplate template = new
RabbitTemplate(connectionFactory);
        template.setMessageConverter(messageConverter());
        return template;
    }
}
```

В pom добавляем:

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
```

4. Задания для самостоятельной работы

Задание 1. Запуск и проверка топологии

- **Убедимся, что Fanout работает корректно.**
- **Задача:**
 - Запустите RabbitMQ, analytics-service, demo-rest, audit-service.
 - Сделайте POST запрос в swagger:



Fanout

Name	Description
id	Integer (32bit)

- Проверьте логи обоих сервисов (demo-rest и audit-service). Оба должны отреагировать.
- Зайдите в RabbitMQ UI (localhost:15672) -> Exchanges -> analytics-fanout. Убедитесь, что в разделе «Bindings» вы видите две разные очереди.

Задание 2. Отказоустойчивость gRPC

- Обработаем недоступность удаленного сервера.
- Задача:
 - Остановите analytics-service.
 - Сделайте запрос на рейтинг в demo-rest. Вы получите исключение.
 - Оберните gRPC вызов в try-catch. В случае ошибки возвращайте рейтинг «-1» и не отправляйте событие в RabbitMQ.

При использовании **Fanout** будьте осторожны с **Routing Key**. Хотя вы можете его передавать (например, в `convertAndSend`), Fanout-обменник его **полностью игнорирует**. Если вам нужно и вещание, и фильтрация одновременно, вам придется использовать Topic Exchange и аккуратно настраивать маски (например, очередь А слушает #, а очередь В слушает created.#).